プログラミング　２０２５年７月２５日　期末試験

教師

テキストの参照 OK。一般論として、不正行為は、大学の規程に即す（つまり、年度内の全ての単位の取り消し、無期停学など）。
解答用紙に、氏名と学籍番号を忘れずに書くこと。
本テストを通じて、どのプログラムも Python プログラムとして書かれている。

問題 1.　（各２点：計３０点）【あ】～【そ】に入る言葉を教科書に従い埋めよ。各項目冒頭のページ数は教科書で掲載されているページ数を表している。

- ８ページ：自分のコンピュータで Python プログラミングができるようにするには、Anaconda と呼ばれるパッケージを利用するのがお薦めだ。パッケージとは、言語の機能を拡張する【あ】やプログラムを開発するための支援システム（【い】と呼ばれる）などをまとめたものだ。
- １５ページ：結果が整数になるか小数になるかは大きな違いだ。Python には整数と小数（正確には【う】）があり、これらは区別しなければならない。たとえば 3.0 と 3 とは根本的に異なるのだ。
- １７ページ：専門用語としては、変数を代入することを変数定義、変数の保持している値のことを【え】、変数の名前を【お】と呼ぶ。
- ２０ページ：わかりやすいプログラムにするためには、多少の説明を追加したくなる。そのような説明書きを【か】と呼ぶ。Python では、#から行末までは何を記述してもプログラムの動作にまったく影響を与えない。そのため、自由に【か】を書くことができる。
- ２０ページ：正しくない入力をしてしまった場合にはエラーが報告されているはずだ。

```
> 65 / (167 / 100) * * 2
    65 / (167 / 100) * * 2
                       ^

[     ]: invalid syntax
```

最もよく見るのはこの【き】だろう。これは、入力したプログラムが文法規則に合っていなかった場合に起こる。

- ２２ページ：一部の名前（たとえば if）は特別な用途に使う（キーワードないし【く】と呼ばれる）ため変数名として使えない。
- ２８ページ：関数の定義は必ず def から始まり、続いてその関数の名前（関数名と呼ばれる。ここでは bmi）が続く。この def は関数を定義していることを表すキーワードである。続くカッコの中味は【け】である。関数は似た処理をさまざまなケースに対して行うものだから、どのようなケースなのかが指定できなければならない。【け】はこのために使う変数で、今回のプログラムなら身長（height）と体重（weight）をこの順に指定するようになっている。複数の【け】はカンマ（,）で区切る。また、【け】のあとのコロン（:）も忘れてはならない。以降はその関数が実際に行う処理の内容だ。この部分で return というキーワードが現れた場合、これによって指定された計算結果を関数全体の最終結果とし（この値を関数の【こ】と呼ぶ）、この関数は終了する。
- ３２ページ：なお、print 等は【あ】読み込みの手続きを行わなくても使うことができる。このような関数は【さ】と呼ばれるものに含まれている。【さ】は自動的に読み込まれ、その中の関数は【あ】名を指定せずとも呼び出すことができる。
- ５３ページ：整数や小数は無数の値を含むのに対し、【し】は「True（真）」「False（偽）」というたった２つだけの値からなる。
- １１５ページ：【す】とは、あるものの説明や表現が、そのもの自身を使って行われている状況の事だ。今回の comb_rec 関数であれば、この関数がどういう動作を行うかの記述の中に、comb_rec 関数が登場しているため、【す】関数となっている。
- １２１ページ：同じ問題であっても、解を求めるのにかかる時間はアルゴリズムによって異なる。この時間のことをアルゴリズムの（時間）【せ】と呼ぶ。ただ、【せ】は入力によって異なることに注意しなければならない。

- １３６ページ：SNS で新規アカウントを作るために、指定したアカウント名の人がすでにいるかどうかを調べたい。どうすればよいだろうか。このような問題を考える場合には、まずはデータをコンピュータ内でどう表現するかを決める必要がある。一番単純な方法は、アカウント名が並んだ配列があるとする方法だ。そうすれば、配列の要素を順番に見てゆけば確認できるだろう。この単純なやり方を【そ】探索と呼ぶ。

問題２.　（各２点：計３４点）各 Python プログラムを対話モードで入力した結果を答えよ。なお、先頭の>はプロンプトを表す。

(2-1)

```
> 2 + 5
```

(2-2)

```
> 13 / 5
```

(2-3)

```
> 10 + 0.0
```

(2-4)

```
> print('5+9.0')
```

(2-5)

```
> print('abc\ndef')
```

(2-6)

```
> def square(x):
>   return x * x
>   print('the square of x')
>
> square(3)
```

(2-7)

```
> a = [8, 5, 3, 2, 4]
> a[2]
```

(2-8)

```
> (1 == 2) and (1 < 2)
```

(2-9)

```
> (1 == 2) and (1 < 2) or (5 == 5 == 5)
```

(2-10)

```
> type(1)
```

(2-11)

```
> type(1.1)
```

(2-12)

```
> type( 1 == 1.0 )
```

(2-13)

```
> type('1')
```

(2-14)

```
> [1, 2] * 3
```

(2-15)

```
> [i // 2 for i in range(0, 5) if i % 3 == 0]
```

(2-16)

```
> def factorial(n):
>   f = 1
>   for i in range(2, n+1):
>     f = f * i
>   return f
>
> factorial(5)
```

(2-17)

```
> def factorial(n):
>   f = 1
>   for i in range(2, n+1):
>     f = f * i
>   return f
>
> factorial(0)
```

問題 3. （１２点）正の整数値 n を与えると、1 から n までの総和を与える関数 souwa を以下の Python プログラムとして作成した。ところが意図通りに動作しなかった。

```
1 > def souwa(n):
2 >     if n <= 0:
3 >         print(0)
4 >     else:
5 >         return n + souwa(n - 1)
```

(3-1) １行だけ修正して意図通りに動作するようにしたい。何行目をどのように修正すれば良いか答えよ。行数はプログラム横に書かれた１～５を用いよ。

問題 4. ((4-1) 1 8点、(4-2) 6点：計2 4点) 各問いに答えよ。

```
> def facE(n):
>  if n == 1:
>   return [[0]]
>  else:
>   res = []
>   for i in range(n):
>    for j in facE(n-1):
>     res.append( j + [i])
>   return res
```

(4-1) facE(3) を求めよ。答えだけで良い。

(4-2) 変数 n が正整数の値 n をもち、変数 a が 0 以上 n! 未満の整数の値をもつとする。そして、リスト facE(n)[a] の値を $[l_0, l_1, \ldots, l_{n-1}]$ と書くことにする。このとき $\sum_{0 \le j \le n-1} j! \times l_j$ を求めよ。答えだけで良い。